

Static Maze Solver

(Search vs. Optimization)

Prepared by:

1- Arwa Khalid

2- Aya Mohammed

3- Ereny Shehata

Section: CS 1

1. Introduction

1.1 Problem Definition

Pathfinding is a fundamental challenge in artificial intelligence and robotics, involving the determination of a valid route from a source to a destination through a space containing obstacles. In this project, the problem is framed as a discrete grid-based maze where an agent must navigate from the top-left corner to the bottom-right corner while avoiding walls.

1.2 Objective of the Project

The primary objective is to implement and compare three classical search algorithms—Breadth-First Search (BFS), Depth-First Search (DFS), and A* and evaluate their performance in terms of speed, optimality, and resource consumption. Furthermore, the project introduces a genetic-inspired optimization phase to investigate how stochastic modifications can refine the generated paths.

1.3 Importance of Pathfinding and Optimization

Efficient path finding is critical in various domains, including autonomous vehicle navigation, video game AI, and logistics. Optimization techniques are equally vital, as finding "any" path is often insufficient; real-world applications demand the most cost-effective or shortest path to conserve time and resources.

2. Problem Formulation

2.1 Maze as a Graph

The maze is modeled as a mathematical graph $G = (V, E)$. Each cell in the 20×20 grid represents a node $v \in V$. Edges E exist between adjacent cells (Up, Down, Left, Right) that are not blocked by obstacles. The graph is unweighted, meaning each move has a uniform cost of 1.

2.2 State Space Representation

- **States:** Coordinate pairs (x, y) where $0 \leq x, y < 20$.
- **Obstacles:** Cells marked as '1' (impassable).
- **Free Space:** Cells marked as '0' (passable).
- **Start State:** $(0, 0)$.
- **Goal State:** $(19, 19)$.

3. Search Algorithms Used

3.1 Breadth-First Search (BFS)

BFS explores the maze level-by-level, using a queue (FIFO) data structure. It explores all neighbors of the current node before moving to the next level of depth.

- **Advantages:** Guarantees the shortest path in unweighted graphs.
- **Disadvantages:** High memory consumption as it stores all nodes at the current depth.

3.2 Depth-First Search (DFS)

DFS explores as far as possible along each branch before backtracking, using a stack (LIFO) data structure.

- **Advantages:** Lower memory requirements than BFS in deep graphs.
- **Disadvantages:** Not optimal; it may find a very long, convoluted path even if a shorter one exists.

3.3 A* Search

A* is an informed search algorithm that uses both the actual distance from the start $g(n)$ and an estimated distance to the goal $h(n)$.

- **Advantages:** Highly efficient and optimal (with an admissible heuristic).
- **Disadvantages:** Complexity depends heavily on the quality of the heuristic.

3.4 Comparative Summary

Algorithm	Optimality	Time Complexity	Memory Usage	Suitability
BFS	Yes	$O(V + E)$	High	Shortest path in small grids
DFS	No	$O(V + E)$	Moderate	General connectivity tasks
A*	Yes	$O(E \log V)$	Moderate/High	Efficient real-time navigation

4. Heuristic Function (A*)

The A* implementation utilizes the **Manhattan Distance** as its heuristic $h(n)$. It is calculated as:

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

This heuristic is **admissible** because it never overestimates the distance to the goal (it assumes a straight line ignoring walls). By prioritizing nodes that appear closer to the target, A* significantly reduces the number of nodes visited compared to blind search algorithms like BFS.

5. Genetic-inspired Optimization Component

5.1 Simplified Genetic Approach

Once a path is found by a search algorithm, the system applies a stochastic optimization phase. Unlike a full Genetic Algorithm (GA) which uses a large population and crossover, this system uses a "Hill Climbing" style genetic mutation. It iteratively attempts to improve a single individual (the path).

5.2 Fitness Function

The fitness of a path is defined by the following equation:

$$\text{Fitness} = \text{Path Length} + h(\text{end_node}, \text{goal})$$

This function penalizes longer paths and rewards paths that end closer to the goal. In our specific implementation, since the initial search algorithm already produces a valid path to the goal, the fitness function is mainly used for local comparison between mutated versions of the same path rather than evaluating global exploration quality.

5.3 Mutation Mechanism

The mutation function selects a random internal node in the path and attempts to move it to an adjacent free cell. If the move is valid, it is applied. This introduces small local changes to the path, and improvements are kept only if they reduce the fitness value.

6. Termination Condition

The optimization process terminates after a **fixed number of iterations** (default: 100). This stopping criterion is chosen to ensure predictable execution time. Since the search space in a **20 × 20** grid is relatively small, 100 iterations provide a sufficient window for local improvements without excessive computational overhead.

7. Visualization

The system utilizes the **Pygame** library to provide real-time visual feedback. Visualization is essential for debugging and educational purposes, as it allows the observer to see the "frontier" of the search expand.

- **Black:** Walls/Obstacles.
- **Blue:** Visited nodes (Explored space).
- **Yellow:** Final computed path.
- **Green:** Start node.
- **Goal:** Red node.

8. Performance Metrics

During execution, the system tracks two primary metrics:

1. **Time Taken (seconds):** Measures the efficiency of the algorithm's logic.
2. **Nodes Visited:** Measures the exploration efficiency. A lower number indicates a more "intelligent" search.

Typically, **A*** visits the fewest nodes, while **BFS** visits a large portion of the grid before finding the goal. **DFS** often visits many nodes in a deep, linear fashion before stumbling upon the exit.

9. Discussion

9.1 Algorithm Performance

A* consistently outperforms BFS and DFS because it uses domain-specific knowledge (the heuristic). While BFS is guaranteed to find the shortest path, it does so blindly in all directions. A* focuses its "energy" towards the goal.

9.2 Role of Genetic Optimization

The genetic-inspired optimization improves an already generated path by applying small local mutations. It can help reduce minor inefficiencies in suboptimal paths, while having limited effect on already efficient paths such as those produced by A*.

9.3 Limitations

The current system uses a simplified mutation that does not guarantee path continuity (it may "break" the path if not carefully constrained). Furthermore, the lack of crossover limits the ability to combine "good features" from different paths.

10. Conclusion

This project successfully demonstrates the trade-offs between different search strategies. A* remains the gold standard for grid-based pathfinding due to its efficiency and optimality. The inclusion of a genetic optimization phase illustrates how stochastic methods can refine solutions, although their impact is most significant on suboptimal initial results. The visualization component serves as a powerful tool for verifying the theoretical behaviors of these algorithms in a practical environment.